

UNIT 3 : Adversarial Search and Games

Introduction to Game Theory

Game theory is a way of making decisions when two or more people are competing with each other. In game theory, a person chooses what to do by thinking about all the possible actions the other person might take, not just the possible results. It helps a person decide the best action in a competitive situation.

Game theory does not tell people what they must do. Instead, it explains the rules and methods that help people choose the best action.

A **game** is an activity involving two or more people who follow certain rules. At the end of the game, each person either gains something or loses something. The set of rules defines the game. Going through the set of rules once by the participants defines a play

Optimal Decisions in Games

In game theory, an optimal decision is a decision that gives a player the best possible outcome, assuming that the opponent is also making the best possible decisions for themselves.

1. Objective of Optimal Decisions

The main goal of a player in a game is to:

Maximize gains (profits, payoffs) or

Minimize losses, depending on the situation.

An optimal decision helps a player perform as well as possible no matter what action the opponent chooses.

2. Assumptions in Game Theory

To find optimal decisions, the following assumptions are usually made:

Players are rational (they think logically).

Each player wants to maximize their own benefit.

Each player is aware of the rules of the game and the possible actions of opponents.

3. Payoff

A payoff is the gain or loss received by a player after the game is played.

Payoffs are usually shown in a payoff matrix.

The matrix helps players compare outcomes for different strategies.

4. Strategies

A strategy is a complete plan of action a player chooses.

There are two main types:

Pure strategy: A player chooses one action and sticks to it.

Mixed strategy: A player uses more than one action with certain probabilities.

5. Maximin and Minimax Principle

These principles are used to find optimal decisions:

Maximin principle (used by the maximizing player):

Choose the strategy that gives the maximum of the minimum payoffs.

Minimax principle (used by the minimizing player):

Choose the strategy that gives the minimum of the maximum losses.

6. Saddle Point

A saddle point exists when the maximin value equals the minimax value.

If a saddle point exists:

The game has a stable solution.

Players can use pure strategies.

The value at the saddle point is the value of the game.

7. Games Without Saddle Point

If no saddle point exists:

Players use mixed strategies.

Probabilities are assigned to different strategies to find optimal decisions.

8. Importance of Optimal Decisions

Optimal decision-making:

Helps players handle competitive situations

Reduces risk of large losses

Provides a scientific approach to decision-making in economics, business, politics, and military planning

Heuristic Alpha–Beta Tree Search

1. Introduction

Heuristic Alpha–Beta Tree Search is an **improved version of the Minimax algorithm** used in game playing (like chess, checkers, tic-tac-toe).

It helps in **making optimal decisions** by **reducing the number of nodes** that need to be evaluated in a game tree.

2. Game Tree

A **game tree** represents all possible moves in a game.

Each node represents a **game position**.

Levels of the tree represent alternate moves of two players:

MAX player (tries to maximize the score)

MIN player (tries to minimize the score)

3. Minimax Algorithm (Brief)

Minimax examines **all possible moves** to choose the best one.

It becomes **slow and inefficient** for large games because the number of possible moves grows rapidly.

4. Alpha–Beta Pruning

Alpha–Beta pruning is a technique that **speeds up minimax** by eliminating branches that **cannot affect the final decision**.

Alpha (α)

The **best (highest) value** that the MAX player can guarantee so far.

Beta (β)

The **best (lowest) value** that the MIN player can guarantee so far.

5. Pruning Condition

When $\alpha \geq \beta$, further exploration of that branch is **stopped (pruned)**.

This means the branch cannot produce a better result.

6. Heuristic Evaluation

In real games, it is impossible to search the tree till the end.

A **heuristic evaluation function** is used to estimate the value of a position.

It gives an **approximate score** instead of an exact outcome.

Example:

In chess, heuristics may consider:

Material value

Piece position

King safety

7. Heuristic Alpha–Beta Search Process

Build the game tree up to a **limited depth**.

Use a **heuristic function** to evaluate leaf nodes.

Apply **minimax** logic to propagate values upward.

Use **alpha–beta pruning** to remove unnecessary branches.

Choose the move with the **best evaluated value**.

Monte Carlo Tree Search (MCTS)

1. Introduction

Monte Carlo Tree Search is a **decision-making algorithm** used in **game playing and AI**.

It is especially useful for **large and complex games** where it is difficult to evaluate all possible moves (e.g., Go, Chess, video games).

MCTS finds good moves by **simulating many random games** and learning from their results.

2. Basic Idea

Instead of searching the entire game tree, MCTS:

Builds the tree **gradually**

Uses **random simulations (playouts)**

Improves decisions based on **experience**

3. Four Main Steps of MCTS

MCTS works by repeating the following four steps many times:

1. Selection

Start from the root (current game state).

Select child nodes using a **selection policy** (commonly UCT).

Move down the tree until a node with unexplored moves is found.

2. Expansion

Add one or more **new child nodes** to the tree.

Each child represents a possible move.

3. Simulation (Rollout)

From the new node, play the game **randomly** until the end.

Record the result (win, loss, or draw).

4. Backpropagation

Update the statistics (wins, visits) of all nodes along the path.

Information flows back to the root.

4. UCT Formula (Selection Rule)

The most common selection rule is **UCT (Upper Confidence Bound for Trees)**:

$$UCT_i = \frac{w_i}{n_i} + C \sqrt{\frac{\ln N}{n_i}}$$

Where:

w_i = number of wins for node i

n_i = number of visits to node i

N = total visits of the parent node

C = exploration constant (usually $\sqrt{2}$)

This formula balances:

Exploitation (choosing the best known move)

Exploration (trying less-visited moves)

5. Example: MCTS in Tic-Tac-Toe

Assume it is **Player X's turn**.

Step 1: Selection

The algorithm starts at the current board.

Chooses a move based on UCT values.

Step 2: Expansion

A new move (say, X plays in the center) is added to the tree.

Step 3: Simulation

From this position, both players make **random moves** until the game ends.

Suppose X wins.

Step 4: Backpropagation

The win is recorded:

Visits +1

Wins +1

These values are updated for all nodes on the path.

↻ This process is repeated thousands of times.

6. Final Decision

After many simulations, the algorithm chooses the move with:

Highest number of visits or

Best win rate

In Tic-Tac-Toe, MCTS usually selects the **center move** as optimal.

7. Advantages

Works well for **very large search spaces**

Does not require a complex evaluation function

Improves with more simulations

Can be stopped anytime and still give a good move

8. Limitations

Requires many simulations for accuracy

Random simulations may be weak in some games

Slower for small games compared to minimax

Stochastic Games

What are Stochastic Games?

A **stochastic game** is a game where multiple players take turns making choices. Each decision affects the current situation and what might happen next. Unlike games where everything is predictable, **randomness** is part of the game, meaning things can change in unexpected ways. This makes it harder for players to decide what to do, since they can't always know what will happen next

The essential components of a stochastic game include:

A set of states (S): Represents the possible configurations of the game.

A set of actions (A): The decisions available to each player.

Transition probabilities (P): Random events (e.g., dice rolls) that dictate how the game moves from one state to another.

Reward function (R): The payoff that each player receives based on their actions and the current state.

Discount factor (γ): A value indicating the importance of future rewards, with smaller values placing more emphasis on short-term outcomes.

Partially Observable Games

In a **partially observable game**, players don't have **complete information** about the game state. This means they can't see everything that's happening or know all the details about the other players' choices or the game's environment. Players only have **limited information** based on what they can observe, making it harder to make the best decisions.

Example:

Imagine you're playing a card game, but you can't see the other player's cards. You only know the cards you have and the actions they take. In this case, you're in a **partially observable game** because you don't have all the information to make your decisions.

Key Points:

Incomplete information: Players don't know everything about the game state.

Uncertainty: Players must make decisions based on guesses or assumptions.

Common in real life: Many real-world situations, like poker or business negotiations, are partially observable.

Limitations of Game Search Algorithms

1. Computational Complexity

Problem: As the size of the game tree increases (i.e., the number of possible moves and states), the amount of computation required grows exponentially.

Impact: This makes it **very slow** and **resource-intensive** to search through all possible moves in complex games (like Chess or Go).

Example: In Chess, there are more possible positions than atoms in the universe, so searching every possible move isn't practical.

2. Limited Search Depth

Problem: To make searching faster, most algorithms only look ahead to a **limited depth** in the game tree.

Impact: The algorithm may miss important future moves, leading to **suboptimal decisions**.

Example: In a game like Tic-Tac-Toe, the search depth is manageable, but in complex games like Go, only a few moves can be looked ahead.

3. Imperfect Information

Problem: Many games have **hidden information** (like cards in Poker or the positions of pieces in fog-of-war strategy games).

Impact: Standard search algorithms assume complete information about the game, which makes them **less effective** in such cases.

Example: In Poker, since players can't see each other's cards, a search algorithm won't be able to account for the uncertainty of the opponent's actions.

4. Heuristic Limitations

Problem: Heuristics (evaluation functions) are used to estimate the value of game positions when it's too slow to search the entire tree.

Impact: If the heuristic function is poorly designed or inaccurate, the algorithm can make **bad decisions**.

Example: In Chess, a heuristic might evaluate a position based on material count, but it might ignore things like **piece positioning** or **king safety**, leading to a wrong evaluation.

5. Difficulty with Multi-Agent Games

Problem: In **multi-agent games** (games with more than two players), each player's strategy can change the outcome of the game in unpredictable ways.

Impact: It's hard to model all players' strategies, and the algorithm might **focus only on one player**, missing the interaction between multiple agents.

Example: In a game like **Risk** (a strategy board game), there are many players with different strategies, making it tough to predict outcomes.

6. Dynamic Nature of Games

Problem: Some games change **during play** (like real-time strategy games or games with random events).

Impact: Algorithms that assume a static game state (like turn-based games) may fail to keep up with **rapid changes**.

Example: In real-time games like StarCraft, players have to make decisions **on the fly**, which is difficult for traditional search algorithms.

7. Lack of Adaptability

Problem: Many game search algorithms do not adapt well to new or changing conditions in the game.

Impact: If an opponent changes their strategy, the algorithm might become **predictable** or **inefficient** over time.

Example: A chess-playing algorithm might perform poorly against a new strategy if it hasn't been trained for that specific type of move.

8. Inability to Handle Imperfect Play

Problem: Most search algorithms assume that players always make **optimal moves** (perfect play).

Impact: If a player or opponent doesn't play perfectly, the algorithm can **underperform** because it expects a level of play that doesn't happen in reality.

Example: In games like **Go** or **Poker**, human players make mistakes, and traditional algorithms may not account for that uncertainty.

Constraint Satisfaction Problems (CSP)

A Constraint Satisfaction Problem is a mathematical problem where the solution must meet a number of constraints. In CSP the objective is to assign values to variables such that all the constraints are satisfied. Many AI applications use CSPs to solve decision-making problems that involve managing or arranging resources under strict guidelines. Common applications of CSPs include:

Scheduling: It assigns resources like employees or equipment while respecting time and availability constraints.

Planning: Organize tasks with specific deadlines or sequences.

Resource Allocation: Distributing resources efficiently without overuse

Components of Constraint Satisfaction Problems

CSPs are composed of three key elements:

1. **Variables:** These are the things we need to find values for. Each variable represents something that needs to be decided. For example, in a Sudoku puzzle each empty cell is a variable that needs a number. Variables can be of different types like yes/no choices (Boolean), whole numbers (integers) or categories like colors or names.

2. **Domains:** This is the set of possible values that a variable can have. The domain tells us what values we can choose for each variable. In Sudoku the domain for each cell is the numbers 1 to 9

because each cell must contain one of these numbers. Some domains are small and limited while others can be very large or even infinite.

3. Constraints: These are the rules that restrict how variables can be assigned values. Constraints define which combinations of values are allowed. There are different types of constraints:

Unary constraints apply to a single variable like "this cell cannot be 5".

Binary constraints involve two variables like "these two cells cannot have the same number".

Higher-order constraints involve three or more variables like "each row in Sudoku must have all numbers from 1 to 9 without repetition".

Inference in CSPs (Constraint Satisfaction Problems)

Inference in the context of CSPs is the process of deducing or inferring new information that helps reduce the search space by using the constraints in the problem. The idea is to make smarter decisions about variable assignments, which can make solving the problem more efficient.

What is Inference in CSPs?

In a CSP:

You have variables, each with a domain (a set of possible values it can take).

There are constraints that limit the ways variables can be assigned values.

Inference refers to using the constraints to eliminate impossible values from the domains of variables or to deduce new information that can help you solve the problem faster.

Backtracking Search for CSPs (Constraint Satisfaction Problems)

Backtracking Search is one of the most widely used algorithms to solve **Constraint Satisfaction Problems (CSPs)**. It is a **search-based algorithm** that systematically explores possible assignments of values to variables and **backtracks** when it encounters a situation where no valid solution is possible.

What is Backtracking Search?

Backtracking is a type of **depth-first search** used to explore the solution space of a CSP. It tries to build a solution incrementally by:

Assigning values to variables.

Checking if the current assignment satisfies the constraints.

If the assignment is valid, it moves to the next variable.

If the assignment is invalid, it **backtracks** (i.e., undoes the last decision) and tries a different value for the last assigned variable.

Advantages of Backtracking Search :

Simple and Flexible: Backtracking is easy to implement and works well for many types of CSPs.

Complete Search: If a solution exists, backtracking will always find it (if given enough time).

No Need for Domain-Specific Knowledge: Backtracking can be used on many types of CSPs without needing to customize the algorithm for each one.

Disadvantages of Backtracking Search :

Inefficiency: Without optimization, backtracking can be slow because it may try many wrong assignments before finding the solution or realizing no solution exists.

Exponential Time Complexity: In the worst case, backtracking has to explore all possible assignments, leading to exponential time complexity, especially for large CSPs.

Not Ideal for Large CSPs: It becomes impractical for very large problems, like solving a complex Sudoku puzzle or large-scale scheduling problems, without further improvements like forward checking or constraint propagation.

